

Advanced Python for Data Science

Week(1): Advanced Python Programming

Thursday, 11th June 2026, 09:30-12:45

Dr. Ghadeer Mobasher

Agenda

1. **Generators & Lazy Evaluation**

Efficient computation over large-scale and streaming data

2. **Decorators and Function Wrapping**

Modification of a function's behaviour

3. **Context Managers and Resource Handling**

Resource management matters

4. **Functional Programming**

Core Principles

Big Picture : Motivation

Why these concepts matter together?

Modern Python is not only about syntax, but about:

- Controlling **when** computation happens
- Controlling **how long** state exists
- Controlling **how functions behave**
- **Structuring** computation pipelines

These features enable scalable and maintainable systems.

Core idea: controlling execution, not just writing code

Eager Evaluation: Problem Statement

What goes wrong with eager computation?

- Large datasets do not fit in memory
- Full computation is unnecessary
- Intermediate Storage is wasteful
- **Core concept:**
 - all values are computed immediately
 - All stored in memory

```
# List comprehension is eager, **loads all into memory** Syntax: square brackets
squares_list = [x**2 for x in range(1_000_000)] # list object

# Memory comparison
import sys
print(sys.getsizeof(squares_list)) # ~8 MB
```

✓ 0.1s

8448728

Lazy Evaluation Idea

Core Concept:

- Compute values only when needed
- Do not store full dataset
- Generate on Demand

```
# Generator Expression - Like list comprehensions, but lazy.
```

```
# Generator expression is lazy, **computes on demand** Syntax: parentheses  
squares_gen = (x**2 for x in range(1_000_000)) # generator object
```

```
# Memory comparison
```

```
import sys
```

```
print(sys.getsizeof(squares_gen)) # ~200 bytes
```

✓ 0.0s

208

Yield-based Generator

```
# Defines a generator function
def even_numbers(limit):
    for i in range(limit):
        if i % 2 == 0:
            ''' Instead of returning a value and exiting,
            this yields the current even number and pauses execution until the next value is requested.'''
            yield i
# Using the generator function
for num in even_numbers(10):
    print(num)
```

```
0
2
4
6
8
```

- A **generator** is a function that:
- Uses **yield** instead of **return**
- Produces values **lazily (on demand)**
- Maintains internal state between iterations
- Is memory-efficient for large datasets

Why it matters?

- Memory Efficiency
- Supports infinite streams
- Enables pipeline



In **Week1.ipynb**, navigate to the section "**Generators and Lazy Evaluation**" and experiment with the code in the cells below.

You have **10 minutes** to do so with your work buddy and discuss it together.

Motivation

Why Decorators matter?

- repeated cross-cutting concerns:
 - logging
 - timing
 - validation
- Goal: **avoid modifying** core logic

Function as an object & Manual Wrapping

```
def greet():  
    print("Hello")
```

```
f = greet  
f()
```

✓ 0.0s

Hello

```
def greet():  
    print("Hello")
```

```
def wrapper():  
    print("Before")  
    greet()  
    print("After")
```

```
wrapper()
```

✓ 0.0s

Before
Hello
After

Decorator Syntax

```
def decorator(func):  
    def wrapper():  
        print("Before")  
        func()  
        print("After")  
    return wrapper  
  
@decorator  
def greet():  
    print("Hello")  
  
greet()  
✓ 0.0s
```

Before
Hello
After

— Key Idea:

- Function transformation
- Separation of concerns
- Reusable Behaviour Injection

Why it matters?

- Frameworks rely heavily on decorators
- Clean architecture on APIs
- Modular behaviour extension



In **Week1.ipynb**, navigate to the section “**Decorators and Function Wrapping**” and experiment with the code in the cells below.

You have **10 minutes** to do so with your work buddy and discuss it together.

Exercises

Solve Problems (1-4) in "**Week1_Exercise.ipynb**" and you have 35 minutes to finish all the tasks. **Don't forget to split your screen. Read the whole instructions first.**

Problem 1: Lazy Evaluation with Generators

A generator produces values one at a time instead of storing the entire result in memory.

Task

1. Write a generator function `count_up_to(n)` that yields numbers from 1 to `n`.
2. Use a `for` loop to print the generated values.
3. Print the type of the object returned by the generator function.

Expected behaviour

Example:

```
count_up_to(5)
```

Output:

```
1 2 3 4 5
```

Reflection Questions

- Why is a generator more memory efficient than returning a full list?
- When would you design a function as a generator rather than returning a collection?

You can work in pairs but remember you have to submit the notebook on Moodle by the end of the lecture

Pause / Break

See you in 15 minutes 😊!
Please be on time.

Problem

```
f = open("file.txt")  
data = f.read()  
f.close()
```

Exception Safety Problem

- exceptions bypass `close()`
- resources remain open
- system instability risk

With Statements

setup → **execution** → **teardown**

- Automatic cleanup
- Safe under exceptions

```
with open("file.txt", "r") as f:  
    data = f.read()
```

Custom Context Manager

— Key Idea:

- A context manager allows Python to automatically perform setup before a block of code and cleanup after the block, **even if an exception occurs.**

```
• # Define a context manager class
class Demo:

    # Automatically called when entering the 'with' block
    def __enter__(self):
        print("Enter")      # Setup operation
        return self        # Return the current object

    # Automatically called when leaving the 'with' block
    def __exit__(self, exc_type, exc_val, exc_tb):
        # exc_type: type of exception (if any)
        # exc_val : exception object (if any)
        # exc_tb  : traceback information (if any)
        print("Exit")      # Cleanup operation

# Create a Demo object
# Call __enter__()
# Execute the indented block
# Call __exit__() when finished
with Demo():

    # Code inside the context manager
    print("Inside")

✓ 0.0s
```

```
Enter
Inside
Exit
```


Why it matters?

- Reliable resource handling
- Safer systems
- Clean abstraction for lifecycle management



In **Week1.ipynb**, navigate to the section "**Context Managers and Resource Handling**" and experiment with the code in the cells below.

You have **10 minutes** to do so with your work buddy and discuss it together.

Core Idea

Functional Programming:

- is a programming paradigm in which programs are constructed by composing functions and avoiding changes to data and state.

Pure Functions: Computation as Function Application

— What Is a Pure Function?

- Produces the same output for the same input.
- Has no side effects (does not modify external variables, files, databases, etc.).

```
def square(x):  
    return x ** 2  
  
print(square(4))  
print(square(4))
```

✓ 0.0s

16
16

Every call with 4 **produces the same result.**

Impure Function Example

```
counter = 0

def increment():
    global counter
    counter += 1
    return counter

print(increment())
print(increment())
```

✓ 0.0s

1
2

The result depends on external state (counter).

A pure function behaves like a **mathematical function**: $f(x) = x^2$, **Input=4 Output=16**

Avoid Mutable State

— What Is Mutable State?

- Mutable state refers to data that can change after it is created.

— Functional Style

- Instead of modifying existing data, create new data.

```
total = 0  
  
for x in [1, 2, 3]:  
    total += x
```

The variable `total` changes repeatedly.

```
sum([1, 2, 3])  
✓ 0.0s  
6
```

No mutable state is required.

Build Pipelines

- Functional programming **often chains operations together.**
- Data flows through a **sequence of transformations.**

```
numbers = [1, 2, 3, 4, 5]

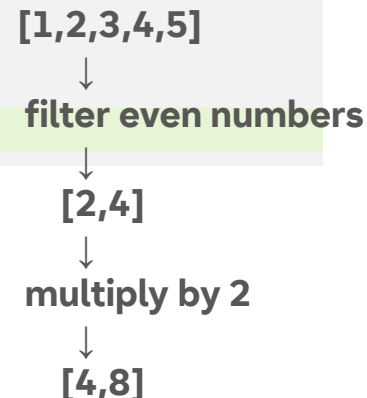
# filter step: keep only even numbers from the list
even_numbers = filter(lambda x: x % 2 == 0, numbers)

# map step: multiply each filtered (even) number by 2
result = map(
    lambda x: x * 2, # transformation function applied to each element
    even_numbers    # iterable containing only even numbers
)

# convert the lazy map object into a list and print it
print(list(result))
```

Pipeline Approach

This style makes programs more predictable, composable, and easier to maintain.



```
numbers = [1, 2, 3, 4, 5]

result = []

for n in numbers:
    if n % 2 == 0:
        result.append(n * 2)

print(result)
```

✓ 0.0s

`[4, 8]`

Traditional Approach

Common Functional Tools

```
from functools import reduce
```

```
nums = [1, 2, 3, 4]
```

```
print(list(map(lambda x: x * 2, nums)))
```

```
print(list(filter(lambda x: x % 2 == 0, nums)))
```

```
print(reduce(lambda a, b: a + b, nums))
```

✓ 0.0s

```
[2, 4, 6, 8]
```

```
[2, 4]
```

```
10
```

Function	Purpose
map()	Transform data
filter()	Select data
reduce()	Combine data
lambda	Anonymous functions
List comprehensions	Concise transformations
Generators	Lazy pipelines



In **Week1.ipynb**, navigate to the section “**Functional Programming**” and experiment with the code in the cells below.

You have **15 minutes** to do so with your work buddy and discuss it together.

Exercises

Solve Problems (5-7) in “**Week1_Exercise.ipynb**” and you have 30 minutes to finish all the tasks. **Don't forget to split your screen. Read the whole instructions first.**

Problem 5: Context Managers

Context managers control resource acquisition and release.

Task

Create a context manager called `managed_resource(name)` using `contextlib.contextmanager`

The behaviour should be:

Opening Working with Closing

Example usage

```
with managed_resource("database") as db:
    print("Working with", db)
```

Question

Why are context managers useful for handling resources like files or database connections?

You can work in pairs but remember you have to submit the notebook on Moodle by the end of the lecture

Problem 6: Performance Monitoring

Unifying Idea

All four concepts control computation:

- **generators** → when values are computed
- **decorators** → how functions behave
- **context managers** → lifecycle of resources
- **functional programming** → structure of computation

Python = composition of controlled execution layers:

- execution flow
- resource flow
- data flow
- function transformation

These abstractions enable:

- scalable systems
- clean architecture
- efficient computation
- production-grade Python code

Take-Home Assignment

Complete the following exercises at home. Remember to split your screen and solve the problems. You may always Google, check the syntax, resolve errors, and brainstorm, but **do not generate code or answers**. See the [submission instructions](#).

Problem 1: Data Transformation Pipeline using `map`, `filter`, and `lambda`

Consider a system that processes a stream of sensor readings collected from multiple devices. Each reading is represented as a tuple `(device_id, value)`.

Some readings are invalid (negative values) and must be discarded. The remaining values must be normalised and transformed for further analysis.

Scenario

Given a list of readings:

```
readings = [  
    ("A", 10),  
    ("B", -3),  
    ("A", 25),  
    ("C", 0),  
    ("B", 15)  
]
```

- The deadline is 17.06.2026 @ 23:59.
- See you next time when we dive into **Concurrency & Asynchronous Workflows**

Any Questions?

See you next time 😊

Dr. Ghadeer Mobasher

ghadeer.mobasher@srh-hochschulen.de

<https://www.linkedin.com/in/ghadeermobasher/>

LGS 6, 2. OG, Arc 213

